

Autonomous Codebase Architecture Mapper

Background

Modern software systems are becoming increasingly complex.

Enterprise applications may consist of thousands of files, multiple repositories, microservices, APIs, databases, external integrations, background jobs, and shared libraries. Over time, the architecture evolves faster than its documentation.

Developers frequently need to understand unfamiliar code before making changes. A seemingly small modification to a function, API, database entity, or service can have unexpected consequences across the system.

In many organizations, understanding these dependencies depends heavily on manual code exploration and knowledge held by experienced developers.

The Problem

Software teams often struggle to answer a fundamental question:

“If I change this piece of code, what else could be affected and why?”

Answering this question may require developers to manually inspect:

- Function and method calls
- Module dependencies
- Service-to-service communication
- API relationships
- Database interactions
- Event and message flows
- Background jobs
- Test coverage
- External integrations

This investigation can be slow, incomplete, and highly dependent on individual experience.

Traditional code analysis tools can identify syntactic relationships, but they may not always provide a complete, understandable representation of how components interact at an architectural level. Documentation, meanwhile, can quickly become outdated as the codebase evolves.

As a result, organizations face challenges such as:

- Longer onboarding time for developers
- Increased risk when modifying unfamiliar code
- Unexpected production regressions
- Difficulty maintaining legacy systems
- Hidden architectural dependencies

- Increased code review effort
- Difficulty identifying critical components
- Inefficient testing after changes

The challenge is to transform a repository from a collection of files into an **understandable, continuously explorable representation of the software system and its relationships**.

Target Users and Organizations

This solution can be valuable for:

- Software developers
- Technical architects
- Engineering teams
- Platform engineering teams
- DevOps and SRE teams
- Engineering managers
- Developers working with legacy systems
- Developers joining unfamiliar projects
- Organizations managing large repositories or microservice architectures

The solution should have the potential to become a **developer intelligence and software architecture product** for organizations.

Challenge

Build an intelligent system that can analyze a software repository and generate an actionable understanding of its architecture and dependencies.

The system should go beyond displaying files and import statements. It should attempt to understand meaningful relationships between components and help users explore how different parts of the software system are connected.

Most importantly, the system should enable a developer to investigate the potential consequences of changing a specific component.

The objective is not simply to generate a diagram.

What does this component do, what does it depend on, what depends on it, and what could be affected if it changes?

Expected Inputs

The system may receive one or more software-related inputs. Possible inputs include:

- Source code repositories
- Individual projects or modules
- Monorepositories
- Multiple related repositories

- API specifications
- Database schemas
- Configuration files
- Dependency manifests
- Test suites
- Infrastructure definitions

Participants may choose which programming languages, frameworks, architectures, and repository structures their solution supports.

Expected Capabilities

A strong solution may demonstrate some or all of the following capabilities. The listed capabilities are intended as guidance and should not limit innovation.

1. Codebase Understanding

The system should analyze the structure of the repository and identify meaningful software components.

- Files
- Modules
- Classes
- Interfaces
- Functions
- Methods
- Services
- Packages
- APIs
- Database entities

Participants are free to determine how deeply their solution analyzes the code.

2. Relationship and Dependency Discovery

The system should identify meaningful relationships between software components.

Function
↓
Service
↓
API
↓
Database

- Function calls
- Method calls
- Imports
- Module dependencies
- API calls
- Service-to-service communication
- Database access
- Event publishing and consumption
- Shared libraries
- External integrations

The exact relationships supported are left open to participants.

3. Architecture Mapping

The system should generate an understandable representation of how the codebase is organized and connected.

- Dependency graphs
- Call graphs
- Service maps
- API interaction graphs
- Database relationship graphs
- Knowledge graphs
- Hierarchical architecture maps

The representation should help users navigate and understand the system rather than simply displaying a large collection of nodes.

4. Architecture and Dependency Insights

The solution may identify meaningful characteristics or potential concerns within the codebase.

- Circular dependencies
- Highly coupled modules
- Critical dependency paths
- Dependency bottlenecks
- Highly connected components
- Isolated modules
- Potential architectural hotspots
- Components with large downstream impact

Core Challenge: Change Impact Analysis

A key capability of the solution should be the ability to investigate the potential impact of changing a selected component.

The user may select a:

- Function
- Method
- Class
- Module
- API
- Service
- Database entity

The system should analyze available relationships and identify components that may potentially be affected.

Example Scenario

A developer selects:

processPayment()

The system may identify:

Potential Impact

12 APIs

4 Database Tables

3 Microservices

27 Tests

However, the value of the system should not stop at providing numbers. The user should be able to understand **why** something is considered impacted.

processPayment()

↓

calls validateTransaction()

↓

used by PaymentService

↓

exposed through /payments API

↓

consumed by Order Service

Risk and Criticality Analysis

Advanced solutions may identify components that represent a higher potential engineering or operational risk. Possible signals may include:

- Number of dependencies
- Number of downstream consumers
- Centrality within the dependency graph
- Code complexity
- Frequency of changes
- Database interactions
- External dependencies
- Test coverage
- Critical execution paths

Participants are free to determine how risk, criticality, or impact should be calculated. A solution does not need to use a predefined scoring formula.

Expected Outputs

The solution should generate **actionable architecture and change-impact insights**.

Architecture Insights

- Repository structure
- Service relationships
- Module dependencies
- API relationships
- Database dependencies
- Dependency paths

Architecture Issues

- Circular dependencies
- Highly coupled components
- Dependency bottlenecks
- Critical components
- Architectural hotspots

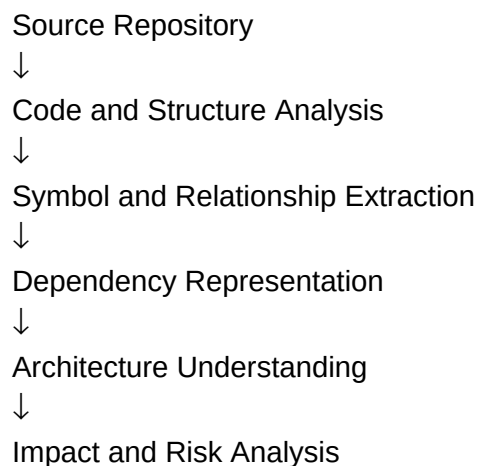
Change Impact Insights

- Potentially affected functions
- Affected modules
- Related APIs
- Connected services
- Database entities involved
- Relevant tests
- Dependency paths explaining the impact

The output format is intentionally open. Participants may design interactive architecture maps, graph visualizations, developer dashboards, IDE integrations, AI-powered exploration interfaces, command-line tools, or automated reports.

Example Processing Pipeline

The implementation approach is intentionally open. One possible approach could be:





Actionable Developer Insights

Participants are free to replace, extend, or redesign this pipeline.

- AST parsing
- Static analysis
- Semantic code analysis
- Graph algorithms
- Knowledge graphs
- LLM-based code understanding
- Code embeddings
- Repository history analysis
- Runtime analysis
- Agentic AI

Scope

Participants are free to decide:

- Programming languages supported
- Frameworks supported
- Repository types supported
- Depth of code analysis
- Types of relationships extracted
- Architecture representation
- Change impact methodology
- Risk analysis methodology
- User interface
- AI models and technologies used

Teams may use open-source, synthetic, private, or self-created repositories for demonstration where permitted.

Constraints

1. The Solution Must Analyze Meaningful Code Relationships

The solution should go beyond basic file exploration, folder visualization, repository summaries, or import listing. The system should demonstrate meaningful understanding of relationships within the software system.

2. AI Should Provide Meaningful Value

If AI or LLMs are used, they should provide value beyond generating summaries of source code. AI may improve semantic code understanding, relationship interpretation, architecture understanding, impact analysis, dependency explanation, or identification of non-obvious relationships.

3. Impact Analysis Must Be Explainable

A list of potentially affected files alone should not be considered sufficient. Where possible, the solution should help the user understand why a component is potentially affected through dependency paths, explanations, relationship graphs, or another approach.

4. Scalability Should Be Considered

Software repositories can contain thousands of files and millions of lines of code. Participants should consider efficient parsing, incremental analysis, graph scalability, processing time, storage requirements, LLM consumption, and computational cost.

5. Static Analysis Alone Is Not the Only Approach

Participants are encouraged to innovate beyond traditional dependency analysis. The challenge intentionally allows different approaches to discovering and understanding relationships.

6. Freedom of Implementation

Participants are not restricted to any particular programming language, AI model, LLM, static analysis framework, graph database, agent framework, visualization technology, or architecture methodology.

Success Criteria and Evaluation

Criteria	What It Measures
Code Understanding	Depth of meaningful understanding of the repository
Relationship Discovery	Ability to identify useful software relationships
Architecture Insight	Quality and usefulness of the architecture representation
Change Impact Analysis	Ability to identify potential consequences of modifications
Explainability	Ability to explain why components are related or impacted
Actionability	How useful the insights are for developers
Technical Quality	Architecture, robustness, and implementation quality
AI Value	Whether AI provides meaningful advantages where used
Efficiency	Processing time, model usage, and computational cost
Scalability	Potential to handle increasingly complex repositories
Innovation	Novelty of the technical approach
Product Potential	Ability to evolve into a practical organizational product
Demo Quality	Ability to clearly demonstrate the working solution

Freedom to Innovate

Participants are free to define their own approach to understanding and mapping software systems. They may use any combination of:

- AST parsing
- Static analysis
- Dynamic analysis
- LLMs
- Code embeddings
- Knowledge graphs
- Graph algorithms
- Agentic AI
- Repository history
- Runtime information
- Custom parsers
- Custom analysis algorithms

The examples provided in this problem statement are illustrative and should not restrict innovation.

Participants are encouraged to introduce additional capabilities that improve software understanding, architectural visibility, developer productivity, or change safety.

Core Objective: Build a system that transforms an unfamiliar codebase into an understandable representation of its architecture and relationships, enabling developers to explore the system and investigate the potential impact of changes before they are made.